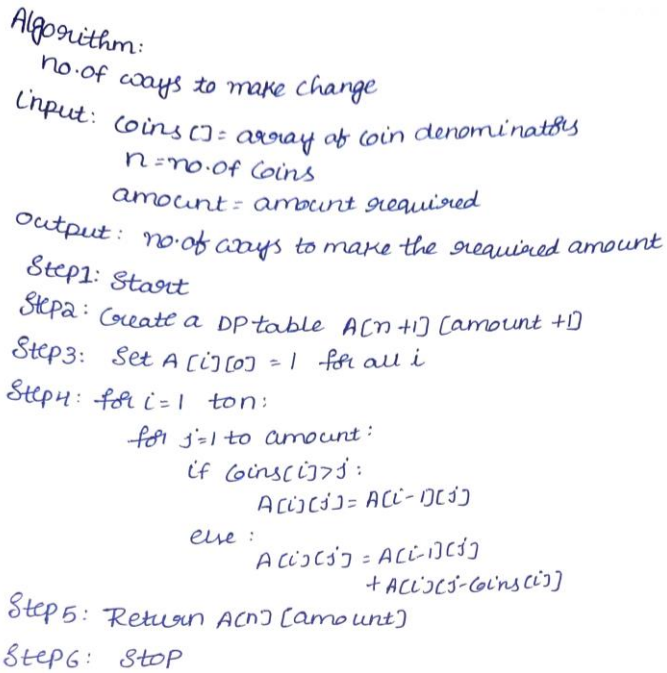
PRACTICAL-7

Aim: To Implementation of making a change problem using dynamic programming

Algorithm:



Code:

```
#include <iostream>
using namespace std;

class CoinChangeWays {

private:
    int coins[10];       // coin denominations
    int n;               // number of coins
    int amount;          // required amount
    int dp[10][100];     // DP table

public:

    // ── Get Input ──
    void getInput() {

        cout << "Enter number of coins   : ";
        cin >> n;
```

```
    cout << "Enter required amount   : ";
    cin >> amount;

    for(int i = 1; i <= n; i++) {
        cout << "Coin " << i << " : ";
        cin >> coins[i];
    }
}

// ── Build DP table ──
void solve() {

    // There is exactly 1 way to make amount 0
    for(int i = 0; i <= n; i++)
        dp[i][0] = 1;

    // With 0 coins, positive amount cannot be made
    for(int j = 1; j <= amount; j++)
        dp[0][j] = 0;

    // Fill DP table
    for(int i = 1; i <= n; i++) {

        for(int j = 1; j <= amount; j++) {

            if(coins[i] <= j) {

                // Exclude + Include
                dp[i][j] =
                    dp[i-1][j]
                    + dp[i][j-coins[i]];

            } else {

                // Cannot include current coin
                dp[i][j] = dp[i-1][j];
            }
        }
    }
```

```
    }

    // ── Print DP table ──
    void printTable() {

        cout << "\nDP Table:" << endl;

        cout << "       ";

        for(int j = 0; j <= amount; j++)
            cout << j << "   ";

        cout << endl;

        for(int i = 0; i <= n; i++) {

            cout << "i=" << i << "   ";

            for(int j = 0; j <= amount; j++)
                cout << dp[i][j] << "   ";

            cout << endl;
        }
    }

    // ── Show Result ──
    void showResult() {

        cout << "\nNumber of Ways = "
             << dp[n][amount] << endl;
    }
};


// ── MAIN ──
int main() {

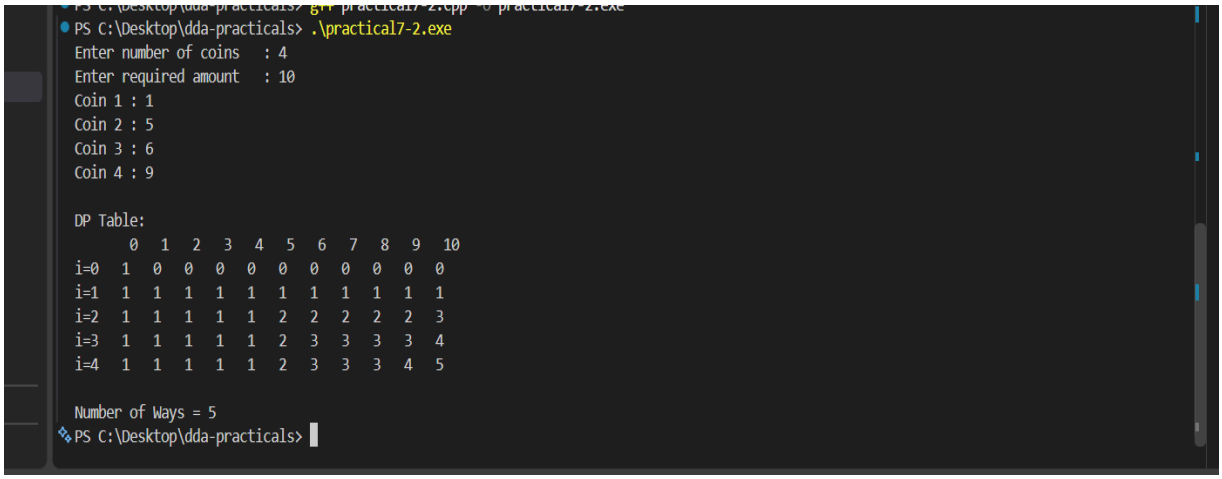
    CoinChangeWays c;

    c.getInput();
```

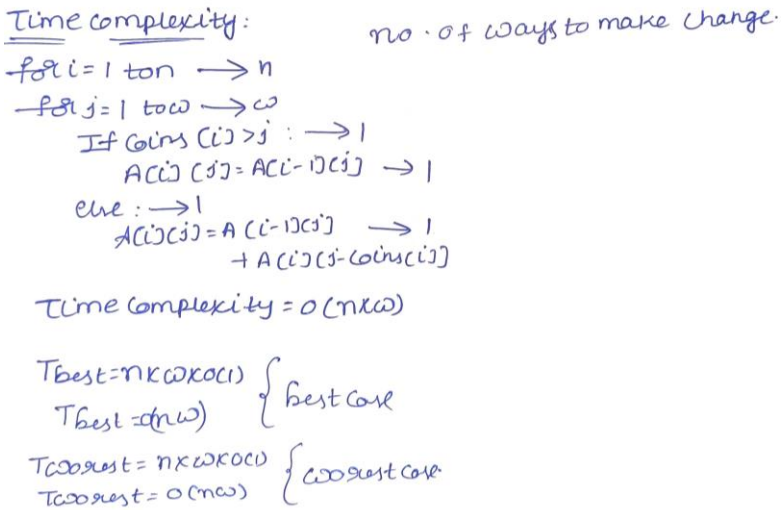
```
    c.solve();
    c.printTable();
    c.showResult();

    return 0;
}
```

Output:



Time complexity:



Algorithm:

Algorithm: minimum Coin Change
input: Coins[] = array of Coins denominations
n = no. of Coins
amount = required amount
output: minimum number of Coins required
Step 1: Start
Step 2: Creat a DP table A[n+1][amount+1]
Step 3: Set A[i][0] = 0 for all i
Step 4: for i = 1 to n:
for j = 1 to amount:
if Coins[i] > j:
A[i][j] = A[i-1][j]
else:
A[i][j] = min(A[i+1][j],

Step 5: Return A[n][amount].

Step 6: Stop.

Code:

```
#include <iostream>
using namespace std;

class CoinChangeMin {

private:
    int coins[10];       // coin denominations
    int n;               // number of coins
    int amount;          // required amount
    int dp[10][100];     // DP table

public:

    // ── Get Input ──
    void getInput() {

        cout << "Enter number of coins   : ";
        cin >> n;

        cout << "Enter required amount   : ";
        cin >> amount;
```

```
    for(int i = 1; i <= n; i++) {
        cout << "Coin " << i << " : ";
        cin >> coins[i];
    }
}

// ── Build DP table ──
void solve() {

    // Base case
    // 0 coins needed to make amount 0
    dp[0][0] = 0;

    // With 0 coins, positive amount is impossible
    for(int j = 1; j <= amount; j++)
        dp[0][j] = 9999;

    // Fill DP table
    for(int i = 1; i <= n; i++) {

        dp[i][0] = 0;

        for(int j = 1; j <= amount; j++) {

            if(coins[i] <= j) {

                // Take or do not take coin
                dp[i][j] = min(
                    dp[i-1][j],
                    1 + dp[i][j-coins[i]]
                );

            } else {

                // Cannot take current coin
                dp[i][j] = dp[i-1][j];
            }
        }
    }
```

```
    }

    // ── Print DP table ──
    void printTable() {

        cout << "\nDP Table:" << endl;

        cout << "       ";

        for(int j = 0; j <= amount; j++)
            cout << j << "   ";

        cout << endl;

        for(int i = 0; i <= n; i++) {

            cout << "i=" << i << "   ";

            for(int j = 0; j <= amount; j++) {

                if(dp[i][j] == 9999)
                    cout << "INF ";
                else
                    cout << dp[i][j] << "   ";
            }

            cout << endl;
        }
    }

    // ── Show Result ──
    void showResult() {

        cout << "\nMinimum Number of Coins = "
             << dp[n][amount] << endl;
    }
};


// ── MAIN ──
```

```
int main() {

    CoinChangeMin c;

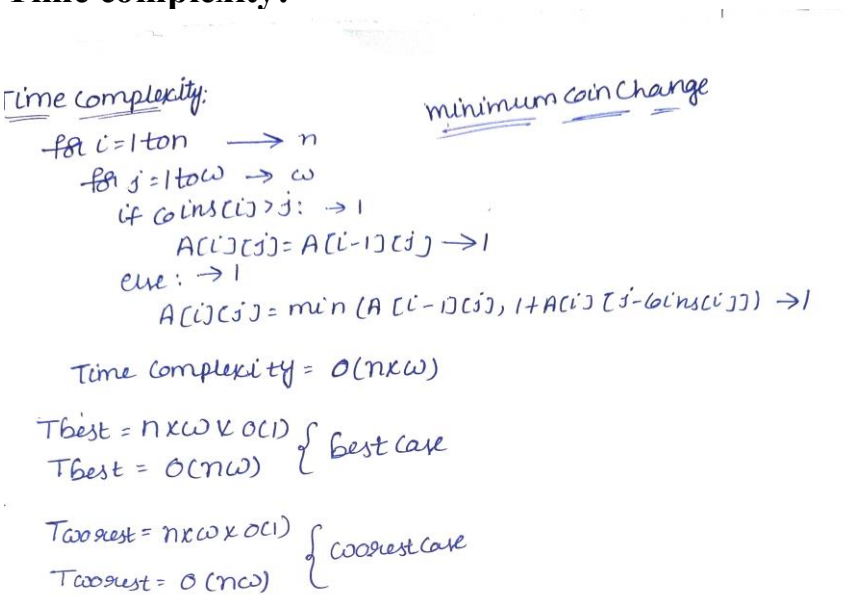
    c.getInput();
    c.solve();
    c.printTable();
    c.showResult();

    return 0;
}
```

Output:



Time complexity:



Result: The Making Change Problem was successfully implemented using Dynamic Programming. The two cases—minimum number of coins and number of ways to make change—were solved efficiently using a DP table. The algorithms have a time complexity of O(n × amount) and demonstrate how dynamic programming can avoid repeated calculations and provide an optimal solution.